

AI Attendance System – Python Code

```
import cv2
import pickle
import face_recognition
from datetime import datetime
import sqlite3

# -----
# CONFIGURATION
# -----

MODEL_PATH = "models/encodings.pkl"
DATABASE_PATH = "database/attendance.db"

# Process every 2nd frame
FRAME_SKIP = 2

# Recognition tolerance
TOLERANCE = 0.5

# -----
# LOAD TRAINED FACE ENCODINGS
# -----

print("Loading trained face encodings...")

with open(MODEL_PATH, "rb") as file:
    data = pickle.load(file)

known_encodings = data["encodings"]
known_names = data["names"]

print(f"Loaded {len(known_encodings)} face encodings.")

# -----
# DATABASE
# -----

def mark_attendance(name):

    connection = sqlite3.connect(DATABASE_PATH)
    cursor = connection.cursor()

    today = datetime.now().strftime("%Y-%m-%d")
    current_time = datetime.now().strftime("%H:%M:%S")

    # Check whether attendance already exists today
    cursor.execute(
        """
        SELECT id
        FROM attendance
        WHERE name = ?
        AND date = ?
        """,
        (name, today)
    )

    existing_record = cursor.fetchone()

    if existing_record is None:

        cursor.execute(
            """
            INSERT INTO attendance
            (name, date, time)
            VALUES (?, ?, ?)
            """,
            (name, today, current_time)
        )

        connection.commit()

        print(f"✓ Attendance marked for {name}")

        connection.close()

        return True

    connection.close()

    return False

# -----
# START CAMERA
# -----

camera = cv2.VideoCapture(0)
if not camera.isOpened():
```

```

print("ERROR: Could not open webcam.")

exit()

print("\nCamera started.")
print("Look at the camera.")
print("Press Q to quit.\n")

frame_count = 0

# Prevent repeatedly displaying attendance messages
attendance_status = {}

# -----
# MAIN LOOP
# -----

while True:

    success, frame = camera.read()

    if not success:

        print("Could not read camera frame.")
        break

    frame_count += 1

    # -----
    # PROCESS EVERY SECOND FRAME
    # -----

    if frame_count % FRAME_SKIP != 0:

        cv2.imshow("AI Attendance System", frame)

        if cv2.waitKey(1) & 0xFF == ord("q"):
            break

        continue

    # -----
    # RESIZE FRAME
    # -----

    small_frame = cv2.resize(
        frame,
        (0, 0),
        fx=0.25,
        fy=0.25
    )

    # OpenCV uses BGR
    # face_recognition uses RGB

    rgb_small_frame = cv2.cvtColor(
        small_frame,
        cv2.COLOR_BGR2RGB
    )

    # -----
    # DETECT FACES
    # -----

    face_locations = face_recognition.face_locations(
        rgb_small_frame,
        model="hog"
    )

    # -----
    # GENERATE FACE ENCODINGS
    # -----

    face_encodings = face_recognition.face_encodings(
        rgb_small_frame,
        face_locations
    )

    # -----
    # PROCESS EACH FACE
    # -----

    for face_location, face_encoding in zip(
        face_locations,
        face_encodings
    ):

```

```

top, right, bottom, left = face_location

# -----
# COMPARE FACE WITH TRAINED FACES
# -----

face_distances = face_recognition.face_distance(
    known_encodings,
    face_encoding
)

name = "Unknown"

if len(face_distances) > 0:

    best_match_index = face_distances.argmin()

    best_distance = face_distances[
        best_match_index
    ]

    if best_distance < TOLERANCE:

        name = known_names[
            best_match_index
        ]

# -----
# CONVERT COORDINATES BACK TO ORIGINAL SIZE
# -----

top *= 4
right *= 4
bottom *= 4
left *= 4

# -----
# DRAW FACE BOX
# -----

cv2.rectangle(
    frame,
    (left, top),
    (right, bottom),
    (0, 255, 0),
    2
)

# -----
# ATTENDANCE
# -----

if name != "Unknown":

    was_marked = mark_attendance(name)

    if was_marked:

        status = "Attendance Marked"

    else:

        status = "Already Marked Today"

else:

    status = "Unknown Face"

# -----
# DISPLAY NAME
# -----

cv2.putText(
    frame,
    name,
    (left, top - 35),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.8,
    (0, 255, 0),
    2
)

# -----
# DISPLAY ATTENDANCE STATUS
# -----

```

```

        cv2.putText(
            frame,
            status,
            (left, top - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.6,
            (0, 255, 255),
            2
        )

# -----
# DISPLAY CAMERA
# -----

cv2.imshow(
    "AI Attendance System",
    frame
)

# -----
# QUIT
# -----

if cv2.waitKey(1) & 0xFF == ord("q"):
    break

# -----
# CLEANUP
# -----

camera.release()

cv2.destroyAllWindows()

print("\nCamera closed.")
print("Attendance system stopped.")

```